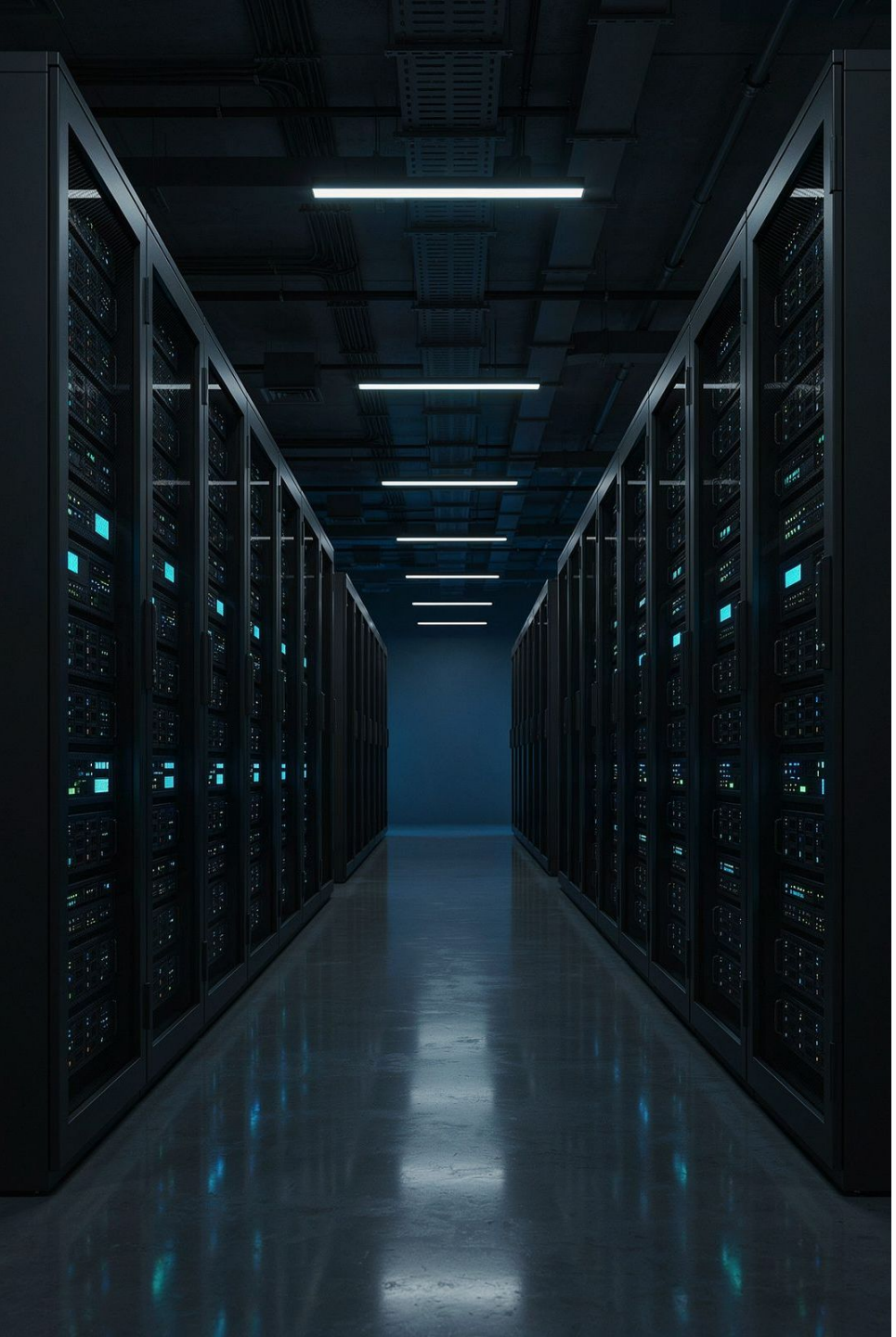


MLOps Platform Engineering

Building production-grade AI infrastructure with SageMaker Pipelines, MLflow, KServe, and CI/CD automation — so your models don't just train, they **ship**.

AWS · MLFLOW · KSERVE · CODEPIPELINE

Why MLOps? The Problem With "It Works on My Laptop"

The Reality Without MLOps

A data scientist trains a fraud-detection model locally. It scores 94% accuracy. Then it reaches production — and nobody knows which version of the code, data, or hyperparameters produced it. Six months later, the model drifts and no one can reproduce the original to debug it.

This is the **ML reproducibility crisis**, and it costs teams months of rework.

What MLOps Solves

- **Reproducibility** — every run is tracked and versioned
- **Automation** — pipelines retrain models on new data without manual effort
- **Governance** — audit trails for compliance and model accountability
- **Speed** — go from experiment to production in hours, not weeks
- **Reliability** — consistent deployment with rollback capabilities

The MLOps Toolchain: An Overview

Think of MLOps like a car manufacturing assembly line — each station has a specific job, and the car (your model) moves smoothly from raw materials (data) to the showroom (production). Here's how our tools map to that pipeline.

1

Experiment Tracking

MLflow logs every run, metric, and artifact

2

Pipeline Orchestration

SageMaker Pipelines automates training workflows

3

Model Registry

SageMaker Registry governs model versions

4

Serving & Inference

KServe deploys models at scale on Kubernetes

5

CI/CD Automation

CodePipeline + CodeBuild ties it all together

SageMaker Pipelines: Automating the Training Workflow

What It Does

SageMaker Pipelines is AWS's managed workflow orchestrator for ML. Instead of manually running data prep, training, and evaluation scripts one by one, you define them as a **directed acyclic graph (DAG)** that executes automatically.

Real-world analogy: Imagine a recipe app that doesn't just list steps — it actually cooks the meal for you, in the right order, every time, and retries if a step fails.

Key Capabilities

- **Step types:** Processing, Training, Evaluation, Condition, Model Registration
- **Conditional branching** — only register a model if accuracy exceeds a threshold
- **Parameterization** — swap datasets or hyperparameters at runtime without code changes
- **Native integration** with S3, ECR, and IAM for secure, scalable runs
- **Full lineage tracking** — know exactly what data and code produced each model



SageMaker Model Registry: Your Model's Source of Truth

Once a pipeline runs, where does the model go? The **SageMaker Model Registry** is a centralized catalog that tracks every trained model version — like a version-controlled library for your ML artifacts.

Version Control for Models

Every model registered includes metadata: training job ARN, dataset version, evaluation metrics, and the pipeline run that created it. No more mystery models in S3.

Approval Workflows

Models move through **Pending → Approved → Rejected** stages. A senior ML engineer or automated evaluation gate must approve before any model reaches production — enforcing governance at scale.

Deployment Trigger

Approval status changes trigger downstream CI/CD events via EventBridge, automatically kicking off a CodePipeline deployment run to push the model to a KServe endpoint.

MLflow: Experiment Tracking Across the Entire Team

The Experiment Chaos Problem

A team of five data scientists is each tuning a recommendation model. They run dozens of experiments — different learning rates, feature sets, architectures. Without tracking, it's **impossible to know which combination worked best**, and results live scattered in notebooks and Slack messages.

MLflow fixes this by providing a unified tracking server where every experiment — parameters, metrics, artifacts, and source code version — is logged automatically and searchable by the whole team.

Core Components

- **Tracking:** Log params, metrics, tags, and model artifacts per run
- **Projects:** Package ML code for reproducible execution
- **Models:** Standard model packaging format (MLmodel) for multi-framework support
- **Registry:** Lightweight model versioning that integrates with SageMaker Registry

MLflow + SageMaker Together

MLflow handles the *experiment phase* — fast, iterative, local or cloud. SageMaker Pipelines handles the *production phase* — scheduled, governed, scalable. The two complement each other perfectly.

> MLflow is framework-agnostic — it works with scikit-learn, PyTorch, TensorFlow, XGBoost, and more, making it the universal experiment layer in any MLOps stack.

KServe: Serving Models at Production Scale

Training a great model is only half the job. **KServe** is the Kubernetes-native model serving platform that handles the other half — deploying models as reliable, scalable inference endpoints that your applications can call in real time.

🚀 Multi-Framework Support

Serves TensorFlow, PyTorch, scikit-learn, XGBoost, ONNX, and custom models through a unified InferenceService API. No custom serving code needed per framework.

⚖️ Autoscaling to Zero

Built on Knative, KServe scales pods based on traffic — including scaling to **zero when idle**, dramatically reducing infrastructure costs for low-traffic models.

🔵 Canary Deployments

Route 10% of traffic to a new model version while 90% hits the stable version. Gradually shift traffic as confidence grows — just like a product A/B test, but for AI inference.

📊 Built-in Observability

Native integration with Prometheus and Grafana for latency, throughput, and error rate monitoring. Drift detection pipelines can be attached per endpoint.

CI/CD for AI: CodePipeline + CodeBuild

In software engineering, no one deploys code by hand anymore — automated CI/CD pipelines do it. **The same principle applies to ML models.** AWS CodePipeline orchestrates the release workflow, and CodeBuild executes the build, test, and deploy steps.

How It Works End-to-End

Trigger: A data scientist merges new training code to the main branch in CodeCommit or GitHub.

CodePipeline detects the change and starts a new pipeline execution automatically.

CodeBuild runs unit tests on the preprocessing and training code, then triggers a SageMaker Pipeline run.

On success, the trained model is registered in SageMaker Model Registry with "Pending Approval" status.

After approval, CodePipeline deploys the model to a KServe InferenceService endpoint in staging, then production.

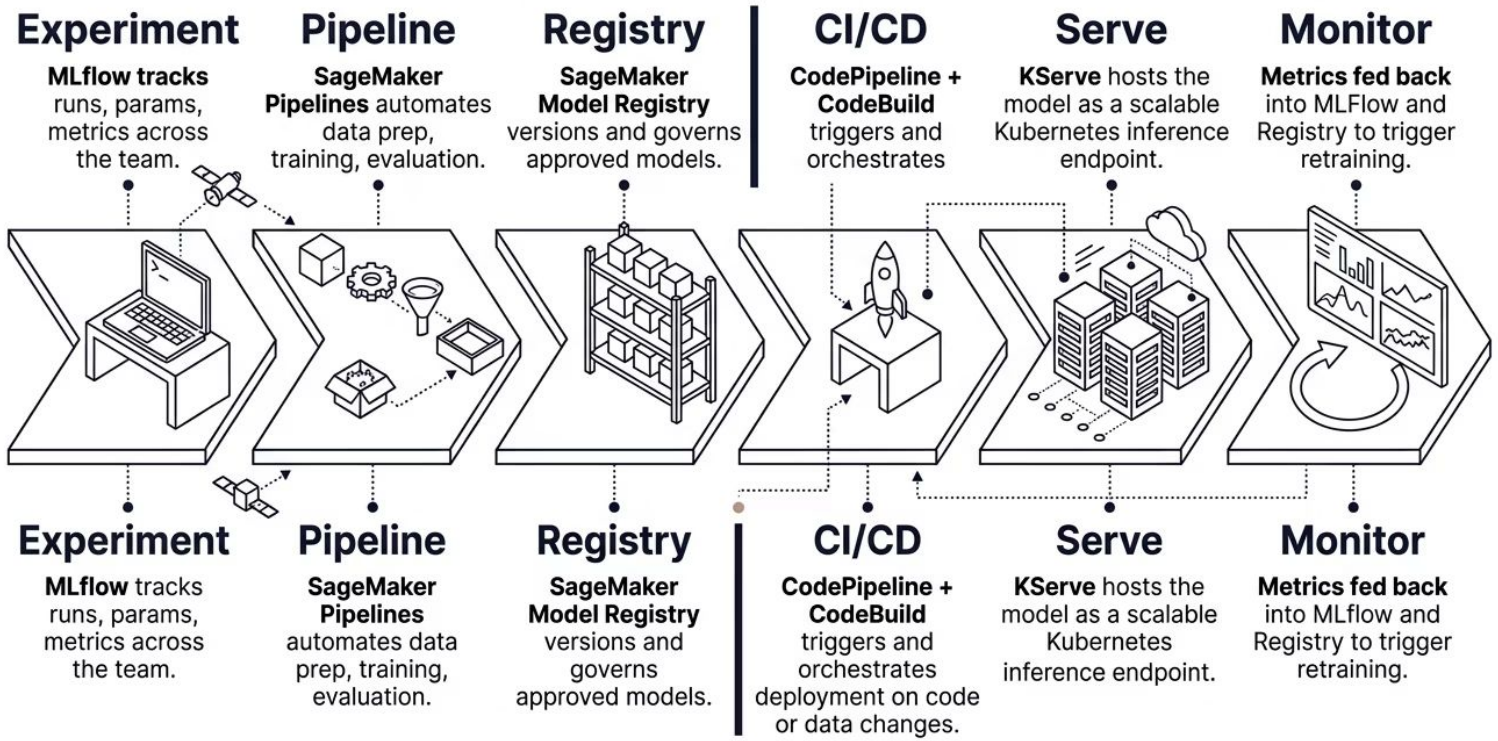
Why This Matters

- **No manual deployments** — humans approve, machines execute
- **Rollback in seconds** — revert to the last approved model version if production degrades
- **Environment parity** — staging and production use identical build artifacts
- **Audit trail** — every deployment is logged with the commit SHA, pipeline run ID, and approver
- **Speed** — a model update that used to take days of manual coordination ships in under an hour

Real example: A retail demand forecasting model gets retrained every Sunday with fresh sales data. CodePipeline handles the entire process automatically — no human intervention required unless metrics fall below threshold.

Putting It All Together: The Full MLOps Lifecycle

Here's how all five tools form a **single cohesive platform** — from a data scientist's first experiment to a model serving millions of requests per day.

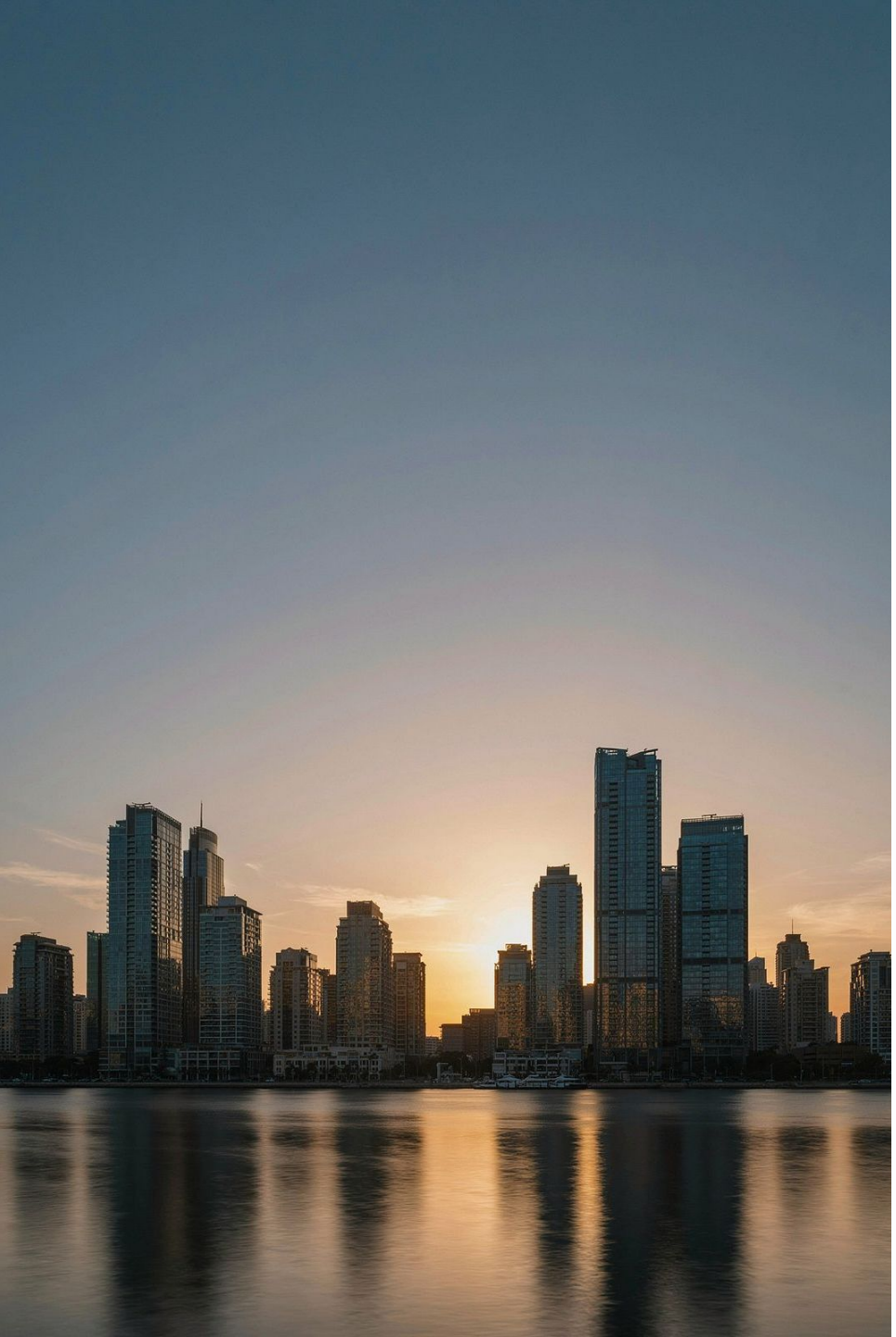


The Business Impact

This platform turns ML from a research activity into a **repeatable engineering discipline**. Models are versioned, deployments are automated, and every decision has an audit trail — meeting both engineering excellence and enterprise compliance requirements.

Key Principle

Each tool has a **single, clear responsibility**. MLflow tracks experiments. SageMaker Pipelines trains at scale. The Registry governs versions. CodePipeline automates releases. KServe serves reliably. No tool overreaches — the platform wins through composition.



Key Takeaways & Next Steps

1

Track Everything with MLflow

Start logging every experiment — parameters, metrics, and artifacts — from day one. Reproducibility is impossible to retrofit later.

2

Automate Training with SageMaker Pipelines

Convert your notebook-based training into a versioned, parameterized pipeline. Add a conditional step to gate model registration on performance thresholds.

3

Govern with the Model Registry

Enforce an approval workflow before any model reaches production. Connect approval events to CodePipeline triggers for hands-free deployment.

4

Deploy Confidently with KServe + CI/CD

Use canary deployments to reduce risk. Let CodePipeline + CodeBuild own the release process so humans can focus on model quality, not deployment mechanics.